10
Logic Programming and core Prolog: paradigm, engine, first examples

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

This week:

- The logic programming (and Prolog) paradigm
- Computational model (resolution + unification)
- Basic, idiomatic programming

Later:

- additional tricks/library of Prolog
- advanced programming, and metaprogramming
- Java/Scala integration with Prolog, by tuProlog

Outline

Logic Programming

Literally: using mathematical logic for computer programming

- logic used as a declarative representation language, and as a theorem-prover (for problem-solving)

However it is comparable to (imperative) programming

- theoretically, one can show Turing completeness
- actually, logic programming is quintessential declarative programming
  - modern functional languages borrow some techniques from logic programming, which is in a sense “even more declarative”
  - typically used to program about planning, searches, rules, symbolic reasoning
- to achieve efficiency/pragmatism, imperative-live mechanisms are used to “control” program execution

Logic Programming mechanisms

Essential/distinctive elements

1. computation is about establishing in how many ways a *goal* can be solved (actually a stream of solutions), with intrinsic trial-and-error exploration of a search space
2. a goal is a relation (either 0-ary, 1-ary, binary, ternary,..) over data elements, which are essentially "untyped trees with holes"

Prolog origins

History

- late 60's problem: of how to represent plans in primitive AI
  - Stanford school: declarative representation, leading to knowledge representation
  - MIT school: procedural representation, leading to functional paradigm
- Knowledge representation languages (Planner, QA-4, ...) evolved to Prolog (1972, A.Colmerauer)

Prolog

- started as theorem prover, now the reference for logic programming (LP)
- ISO Prolog (1995) is still the reference language and library

Prolog in SW Engineering

- very high hype initially, then mostly faded
- now back as the basis for "symbolic AI", not for general programming
- some aspects of Prolog impacted other areas: DBs, XML, pattern matching

Why Prolog in this course?

It covers the "remaining" paradigm: LP (OOP, FP, LP, ...)

- if/when you grasp it, it is fun and easy! (but it needs to be grasped...)
- the "feeling" is completely different from mainstream programming of Java, C, C++, Scala
- programming as "searching into a space of solutions"

Practicing polyglotism (Java/Scala/Prolog)

- Java/Scala as the part handling more "in-the-large" aspects

$\rightarrow$ S/W organisation, connection with the O.S. and libraries

- Prolog as the engine to handle certain data & algorithms

$\rightarrow$ reasoning, space exploration features

A preview: permutations in Structured Programming

Permutations in idiomatic imperative programming (C/Java)

- at each call, the input is updated

```
static boolean nextperm(int[] a){
    int i,k;
    for (i=a.length-2; i>=0 && a[i]>a[i+1]; i--);
    if (i<0){
        return false;
    }
    for (k=a.length-1;a[i]>a[k];k--);
    swap(a,i,k);
    k=0;
    for (int j=i+1;j<(a.length+i)/2+1;j++){
        swap(a,j,a.length-k-1);
        k++;
    }
    return true;
}
```

A preview: permutations in LP

Permutations in idiomatic logic programming (Prolog)

- the goal is to seek any permutation of a list
- `member/3` relates a list with any element in it and the rest of the list
- `permutation/2` relates a list with any permutation of it
  - empty list is permutation of an empty list
  - given a list `L`, let `H` be any element of it, `T` the rest, and `TP` any permutation of `T`, than `L` has as permutation a list starting with `H` and having tail `TP`

```prolog
member([H|T], H, T).
member([H|T], E, [H|T2]):- member(T, E, T2).

permutation([], []).
permutation(L, [H | TP]) :-
    member(L, H, T),
    permutation(T, TP).
```

A preview: permutations in modern FP

Permutations in idiomatic, modern FP (Scala)

- producing a stream of permutations
- solution highly-inspired by idiomatic LP
- `filter` plays the role of `member/3`
- note that LP somewhat inherently deals with "stream of results"

```scala
def member[A](l: List[A]): List[(A, List[A])] = l match
  case a :: Nil => List((a, Nil))
  case a :: t => (a, t) :: (for (a2, l2) <- member(t) yield (a2, a :: l2))

def permutations[A](l: List[A]): Iterable[List[A]] = l match
  case Nil => Iterable(List())
  case _ =>
    for
      (a, l2) <- member(l)
      p <- permutations(l2)
    yield a :: p
```

Conciseness of Prolog

With Prolog you to code certain programs better (more idiomatically) than Java, C, ...

Pros:

- if you master Prolog, you can directly and simply capture desired non-trivial behaviour
- recall that learning a new paradigm means learning new computational patterns
- Prolog syntax and semantics are surprisingly succinct

Cons:

- if you do not correctly understand it, difficulties arise
- there's no true school of clean coding for Prolog
- tooling is very viscose – debugging is very difficult
- incrementality is key to control programs

Prolog Technologies

Free implementations

- Gnuprolog: `http://www.gprolog.org`
- SWIProlog: `http://www.swi-prolog.org`

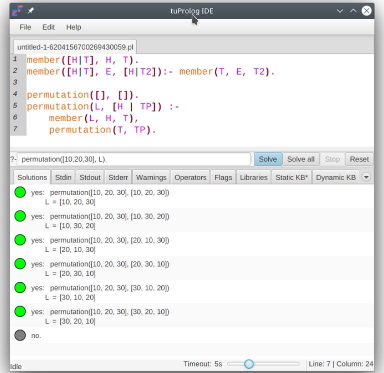
Commercial implementations

- SICStus: `https://sicstus.sics.se`

tuProlog

- an academic open-source framework originally written/combined with Java, developed by "us" (i.e., Ricci + Omicini + ...)
- now rewritten in Kotlin (i.e., Ciatto + Omicini + ...)
- it features library, API, IDE
- we will adopt it, for exercises and polyglotism

tuProlog IDE snapshot and references



References

- 2P-kT: `https://apice.unibo.it/xwiki/bin/view/Tuprolog/`
- 2P-kT web playground: `https://pika-lab.gitlab.io/tuprolog/2p-kt-web/`
- tuProlog 4 (pre 2P-kT, actually more stable): on virtuale

Learning Prolog incrementally

This week: core Prolog

- Two mechanisms: resolution + unification
- Basic goal resolution examples
- Programming with lists

Next Week: full Prolog

- Additional non-core mechanisms
- Additional programming techniques

Later

- Lab with other exercises and tuProlog/Java/Scala integration

Prolog as a programming language: idea

Informal idea: programming as writing a logic theory

- a program is a set of facts, and of rules for deriving facts from other facts
- question/query used to check if (or, at what conditions) the program declares (directly, or indirectly by rules) a certain fact

$\Rightarrow$ essentially, a goal is a candidate "theorem" for the logics of our "axioms/rules"

Example program (two axioms, one general rule):

```
father(abraham, isaac).
father(terach, abraham).
grandfather(GF, GS) :- father(GF, F), father(F, GS).
```

Goals

```
?- father(abraham, isaac).  -> Yes              % 1 solution
?- father(abraham, terach).  -> No              % 0 solutions
?- grandfather(terach, isaac).  -> Yes          % 1 solution
?- father(A, isaac).  -> Yes, A/abraham         % 1 solution
?- grandfather(terach, S).  -> Yes, S/isaac     % 1 solution
?- father(F, S).  -> Yes, F/abraham, S/isaac;   % 2 solutions
                          F/terach, S/abraham
```

Prolog as a programming language: two concepts

Resolution

- computing in Prolog means finding one ore more positive solutions to a "goal" (or list of goals)
- start from first goal G, and find in the program rules whose head matches G, and for each try to solve the body B of the rule (again a list of goals, possibly empty)
- since many rules can match, at each step we have a choice that opens alternatives, all to be explored (via so-called "backtracking")

Unification

- a goal expresses a (mathematical) relation (0-ary, 1-ary, 2-ary, ...) between first-order terms
- terms are the "data values" processed by Prolog: basically, untyped trees with atomic values or logic variables in leaves
- match between terms is done by the "unification algorithm", and gives a substitution that is incrementally refined during solution exploration
- each result of computation is actually a substitution

Outline

Resolution (without matching): syntax

Core abstract syntax of resolution system

$$\texttt{Clause} ::= \texttt{Goal :- } \texttt{Goal}_1, \ldots, \texttt{Goal}_n \qquad (n \geq 0)$$
$$\texttt{Program} ::= \texttt{Clause}_1 \ldots \texttt{Clause}_k \qquad (k > 0)$$

Concrete syntax and additional terminology

- a list of goals is called a resolvent
- a clause has the form "head `:-` body"
- clauses with empty body are called facts, written "`G.`"
- clauses with non-empty body are called rules, written "`G :- G1,..,Gn.`"

Resolution (without matching): semantics

Core abstract syntax of resolution system

$\texttt{Clause} ::= \texttt{Goal :- Goal}_1, \ldots, \texttt{Goal}_n \qquad (n \geq 0)$
$\texttt{Program} ::= \texttt{Clause}_1 \ldots \texttt{Clause}_k \qquad (k > 0)$

Semantics of resolution system: transition relation ("just substitute first head for its body")

- Start with an initial "input" resolvent $\texttt{R}_0 = \texttt{G}_1, \texttt{G}_2, \ldots, \texttt{G}_n$
- A valid computation step ("resolution") is a transition $\texttt{R} \rightarrow \texttt{R}'$, defined as follows
- If clause (fact/rule) $\texttt{G}'$ `:-` $\texttt{G}'_1, \ldots, \texttt{G}'_m$ is defined in `Program`, and $\texttt{G}' = \texttt{G}_1$, then: $\texttt{G}_1, \texttt{G}_2, \ldots, \texttt{G}_n \rightarrow \texttt{G}'_1, \ldots, \texttt{G}'_m, \texttt{G}_2, \ldots, \texttt{G}_n$
- Note that given a `R`, there can be many $\texttt{R}'$ such that $\texttt{R} \rightarrow \texttt{R}'$

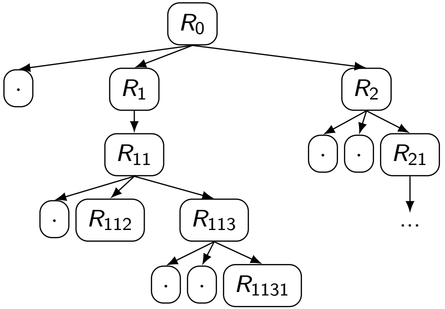
Example resolution

- With clauses:
  - `a.`
  - `a :- b,c.`
- And resolvent: `a,c,d`
- We have only two valid resolutions to go to in one step:
  - $\texttt{a,c,d} \rightarrow \texttt{c,d}$ note we reduced the list of goals
  - $\texttt{a,c,d} \rightarrow \texttt{b,c,c,d}$ note we expanded the list of goals

Resolution (without matching): resolution tree

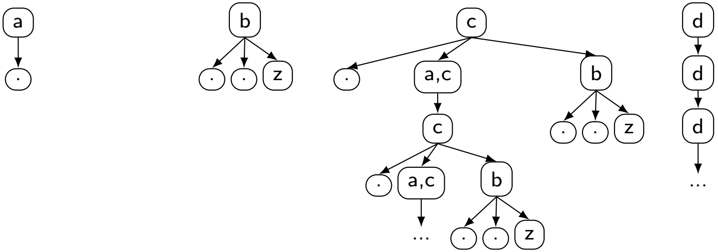
Resolution tree

- For each resolvent, its first goal can match the head of many clauses (always considered from top to bottom), hence several "child" resolvents could be generated
- A resolution is considered successful/complete if there is a leaf with empty resolvent "."
- We generally have a (possibly infinite) tree of resolvents, where solutions are searched left to right (depth first)
- The process of going-up the tree to find a (new) solution is called "backtracking"
- Resolution step: $\mathtt{G}_1, \mathtt{G}_2, \ldots, \mathtt{G}_n \rightarrow \mathtt{G}'_1, \ldots, \mathtt{G}'_m, \mathtt{G}_2, \ldots, \mathtt{G}_n$ if *clause*($\mathtt{G}'$:- $\mathtt{G}'_1, \ldots, \mathtt{G}'_m$) and $\mathtt{G}' = \mathtt{G}_1$



Example program, and resolution trees

```
a.              % a clause with empty body is called a "fact"
b.              % multiple copies of rules/facts can occur
b.
b :- z.         % b is the rule "head", z is the "body"
c.              % different clauses can have same "head"
c :- a, c.      % a sort of recursive rule
c :- b.
d :- d.         % .. recall the order of clauses is relevant
```



Outcomes of resolution: what could we ask?

Predicative

- has it one solution (reaching an empty leaf)? – don't care after first solution

Stream-like

- how many solutions?

Loop-aware

- will computation terminate? (seemingly undecidable)

Output-oriented

- what is the "output" of a solution? is it related to the sequence of resolvents? what is the order of results?

Inference/knowledge with resolutions

Facts

- though they are atomic, they can be used to state knowledge we take as true, e.g., an axiom
- similarly to propositional symbols in propositional logic

Rule

- by a resolvent, we check composition of goals as sort of higher-level knowledge
- a rule is a way of giving such a composition a name (head), and a definition (body)
- essentially: name-based abstraction, with key possibility of recursion

```
father_abraham_isaac.
father_terach_abraham.
grandfather_terach_isaac :- father_abraham_isaac, father_terach_abraham.
```

Shortcomings

Empowering resolution

- goals might have a structure, not just be atomic symbols
- goals seem to express a relationship between "elements"
- might want to express grandfather relation in the general case
- might want to have an explicit notion of "result" (who is abraham's father?)
- need to express computations in a Turing-complete way

Roadmap

- giving a concrete, structured syntax to goals
- providing an advanced mechanism of goal-rule matching
- adding information to the status of computation beyond mere resolvents

$\Rightarrow$ will extend resolution analogously to the transition from "proposition logic" to "first-order logic"

Outline

1. LP intro
2. Basic Resolution
3. Terms and Unification
4. Prolog Resolution
5. Basic programming
6. Programming data types

Empowering resolution framework

Ingredients

- goals are 0-ary, 1-ary, 2-ary, . . . , relations between terms
- terms are the "values" of the Prolog language, and are (finite) trees
- terms can have logic variables in leaves
- goal matching is done by so-called "unification"
  - essentially defined as "a substitution of variables to terms making two goals identical"
  - unification could also provide a declarative form of side-effect
- computation evolves a pair:
  - resolvent
  - substitution "collected so far"
- substitution is considered to be the "result" of computation

Prolog terms

Goals/terms: Prolog syntax

Goal ::= Predicate | Predicate($\texttt{Term}_1$, ..., $\texttt{Term}_n$)

Term ::= Variable | Number | Functor | Functor($\texttt{Term}_1$, ..., $\texttt{Term}_n$)

- Predicates and Functors names are literals starting with lower-case, Variables with upper-case
- Predicates and Functors are said to have arity 0, 1, 2,... n
- Note that syntax of goals is a subset of that of terms
- A term with no variables in it is called *ground*

```
father(abraham, isaac).
father(terach, abraham).
grandfather(GF, GS) :- father(GF, F), father(F, GS).

pred(cons(H, nil), H).  % cons/nil as functors
pred(cons(H, T), L) :- pred(T, L). % what does pred define?

odd(1).            % 1 is odd
odd(3).            % 3 is odd
sum(2, 3, 5).      % 2,3,5 are in the sum relation
```

Interpretations of Prolog programs

Logic interpretation: the classical one

- a program as a "logic theory", computing as "proving a goal is a theorem under that theory"
- e.g.: to prove that `GF` is grandfather of `GS`, first prove that...

Relational interpretation: the idiomatic one – shall use this

- a program as a set of "predicates over terms", computing as "querying for a relation"
- e.g.: `10` is in `element` relation with `cons(10, nil)`

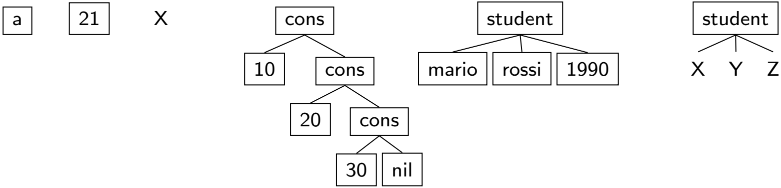
Procedural interpretation: the unproper one

- a program as a "set of procedures", computing as "calling predicates"
- call `element(10, cons(20, nil))` causes call `element(10, nil)`

Terms as trees – terminology and visualisation

A term is a tree of atoms, with atoms/numbers/variables in leafs

- `a`: an atom
- `21`: a number
- `X`: a variable
- `cons(10,cons(20,cons(30,nil)))`: a compound (ground) term
- `student(mario, rossi, 1990)`: a compound (ground) term
- `student(X, Y, Z)`: a compound (non-ground) term



Terms(/Goals) Substitution

Definition

- a substitution $\theta = \{\mathtt{X}_1/\mathtt{T}_1, \ldots, \mathtt{X}_n/\mathtt{T}_n\}$ maps variables to terms
- terms $\mathtt{T}_j$ should contain no variable $\mathtt{X}_k$
- e.g. $\{\}$, $\{\mathtt{Y}/\mathtt{10}\}$, $\{\mathtt{X}/\mathtt{a(1,Z)}, \mathtt{Y}/\mathtt{10}, \mathtt{W}/\mathtt{Z}\}$
- e.g. $\{\mathtt{X}/\mathtt{a(1,Z)}, \mathtt{Y}/\mathtt{10}, \mathtt{Z}/\mathtt{W}\}$ is invalid, should rewrite: $\{\mathtt{X}/\mathtt{a(1,Z)}, \mathtt{Y}/\mathtt{10}, \mathtt{W}/\mathtt{Z}\}$

Related concepts

- application to terms:
  - `p(X,a)`$\{\mathtt{X}/\mathtt{10}\}$ is `p(10,a)`
  - `p(X,a)`$\{\mathtt{Y}/\mathtt{X}\}$ is `p(X,a)` (equivalent to `p(Y,a)` “under that substitution”)
- equivalence of substitutions: $\{\mathtt{X}/\mathtt{Y}, \mathtt{Z}/\mathtt{10}\} \equiv \{\mathtt{Y}/\mathtt{X}, \mathtt{Z}/\mathtt{10}\}$
- generality: $\{\mathtt{X}/\mathtt{10}\}$ more general than $\{\mathtt{X}/\mathtt{10}, \mathtt{Y}/\mathtt{2}\}$
- composition of substitutions:
  - $\{\mathtt{X}/\mathtt{10}\}\{\mathtt{Y}/\mathtt{X}\} \equiv \{\mathtt{X}/\mathtt{10}, \mathtt{Y}/\mathtt{10}\}$
  - $\{\mathtt{X}/\mathtt{10}\}\{\mathtt{X}/\mathtt{5}\}$ impossible
- term/goal instance: `p(10, b)` is an instance of `p(X, b)`
- term cloning: *clone*(`p(10,X,X,Y)`) = `p(10,X2,X2,Y2)`, with (X2, Y2 fresh)

Most general unifier

Definition

- *mgu*($\mathtt{T}_1, \mathtt{T}_2$) is any substitution θ such that:
  - it is a unifier of $\mathtt{T}_1$, and $\mathtt{T}_2$, namely, $\theta\mathtt{T}_1 = \theta\mathtt{T}_2$
  - it is a most general one, namely, out of many unifiers we exclude less general ones
- an *mgu* might not exist, if many *mgu* seem to exist, they are actually all equivalent substitutions

Examples

- *mgu*(`a(1,2)`, `a(X,Y)`) $= \{$`X/1`, `Y/2`$\}$
- *mgu*(`a(1,2)`, `a(X,X)`) $= \bot$
- *mgu*(`a(1,b(X))`, `a(Y,b(Z))`) $= \{$`X/Z`, `Y/1`$\}$
- *mgu*(`a(X,Y,A,b(W))`, `a(Z,Z,B,b(1))`) $= \{$`X/Z`, `Y/Z`, `W/1`, `A/B`$\}$

What is unification

- a symmetrical pattern matching mechanism
- binding variables to non-variable terms, and some other variables into groups
- e.g. in last case above: `W/1`, and groups (`X, Y, Z`) and (`A,B`)
- MGU can be simply tested in Prolog: `?- p(X,1) = p(2,Y). -> yes X/2, Y/1`
- opposite of MGU can be tested by: `?- p(X,1) \= p(2,Y). -> no`

Unification algorithm (by Martelli, Montanari, 1982)

$G \cup \{t \doteq t\} \Rightarrow G$		**delete**
$G \cup \{f(s_0, \ldots, s_k) \doteq f(t_0, \ldots, t_k)\} \Rightarrow G \cup \{s_0 \doteq t_0, \ldots, s_k \doteq t_k\}$		**decompose**
$G \cup \{f(s_0, \ldots, s_k) \doteq g(t_0, \ldots, t_m)\} \Rightarrow \bot$	if $f \neq g$ or $k \neq m$	**conflict**
$G \cup \{f(s_0, \ldots, s_k) \doteq x\} \Rightarrow G \cup \{x \doteq f(s_0, \ldots, s_k)\}$		**swap**
$G \cup \{x \doteq t\} \Rightarrow G\{x \mapsto t\} \cup \{x \doteq t\}$	if $x \notin \mathrm{vars}(t)$ and $x \in \mathrm{vars}(G)$	**eliminate**[note 8]
$G \cup \{x \doteq f(s_0, \ldots, s_k)\} \Rightarrow \bot$	if $x \in \mathrm{vars}(f(s_0, \ldots, s_k))$	**check**

Outline

Resolution (with unification): core Prolog semantics

Semantics of Prolog resolution system: transition relation

- Use a tree of pairs of $\langle$ resolvent $+$ substitution $\rangle$ as computation state
- Start with an initial "input" configuration $C = \langle \mathtt{R}_0 : \{\} \rangle$
- A valid computation step is a transition $\mathtt{C} \rightarrow \mathtt{C}'$, defined as follows
- If $\mathtt{G}'$ `:-` $\mathtt{G}'_1, \ldots, \mathtt{G}'_m$ is the clone of a clause in `Program`, and $\theta' = mgu(\mathtt{G}', \mathtt{G}_1)$ exists, then: $\langle \mathtt{G}_1, \mathtt{G}_2, \ldots, \mathtt{G}_n : \theta \rangle \rightarrow \langle (\mathtt{G}'_1, \ldots, \mathtt{G}'_m, \mathtt{G}_2, \ldots \mathtt{G}_n)\theta' : \theta\theta' \rangle$
- For simplicity, of a θ in a configuration only the part that mentions variables in the resolvent and in the input resolvent is needed

Resolution tree

- The transition relation induces as usual a possibly infinite tree
- A "solution" is the substitution θ^s we have in a leaf with empty resolvent

Resolution with unification: example 1

Resolution step

- If $\texttt{G}' \texttt{ :- } \texttt{G}'_1, \ldots, \texttt{G}'_m$ is the clone of a clause in `Program`, and $\theta' = mgu(\texttt{G}', \texttt{G}_1)$ exists, then:
  $\langle \texttt{G}_1, \texttt{G}_2, \ldots, \texttt{G}_n : \theta \rangle \rightarrow \langle (\texttt{G}'_1, \ldots, \texttt{G}'_m, \texttt{G}_2, \ldots \texttt{G}_n)\theta' : \theta\theta' \rangle$
- For simplicity, of a θ in a configuration only the part that mentions variables in the resolvent and in the input resolvent is needed

```
father(abraham, isaac).
father(terach, abraham).
grandfather(GF, GS) :- father(GF, F), father(F, GS).
```

```
C1:  father(terach, X): {}
-->
C2:  yes : {X/abraham}
```

Explanation

- cloned rule whose head matches the goal: `father(terach, abraham).`
- $\theta' = \{X/abraham\}$
- new resolvent is empty (yes)

Resolution with unification: example 2

Resolution step

- If $\mathtt{G}'$:- $\mathtt{G}'_1, \ldots, \mathtt{G}'_m$ is the clone of a clause in Program, and $\theta' = mgu(\mathtt{G}', \mathtt{G}_1)$ exists, then: $\langle \mathtt{G}_1, \mathtt{G}_2, \ldots, \mathtt{G}_n : \theta \rangle \rightarrow \langle (\mathtt{G}'_1, \ldots, \mathtt{G}'_m, \mathtt{G}_2, \ldots \mathtt{G}_n)\theta' : \theta\theta' \rangle$
- For simplicity, of a θ in a configuration only the part that mentions variables in the resolvent and in the input resolvent is needed

```
father(abraham, isaac).
father(terach, abraham).
grandfather(GF, GS) :- father(GF, F), father(F, GS).
```

```
C1:  father(terach, X), father(X, Y): {}
-->
C2:  father(abraham, Y) : {X/abraham}
```

Explanation

- cloned rule whose head matches the goal: `father(terach, abraham).`
- $\theta' = \{X/abraham\}$
- new resolvent: `father(X, Y)`
- applying θ': `father(abraham, Y)`
- θ' must be preserved since X is used in the goal

Resolution with unification: example 3

Resolution step

- If $\mathtt{G}'$ `:-` $\mathtt{G}'_1, \ldots, \mathtt{G}'_m$ is the clone of a clause in `Program`, and $\theta' = mgu(\mathtt{G}', \mathtt{G}_1)$ exists, then:
  $\langle \mathtt{G}_1, \mathtt{G}_2, \ldots, \mathtt{G}_n : \theta \rangle \rightarrow \langle (\mathtt{G}'_1, \ldots, \mathtt{G}'_m, \mathtt{G}_2, \ldots \mathtt{G}_n)\theta' : \theta\theta' \rangle$
- For simplicity, of a θ in a configuration only the part that mentions variables in the resolvent and in the input resolvent is needed

```
father(abraham, isaac).
father(terach, abraham).
grandfather(GF, GS) :- father(GF, F), father(F, GS).
```

```
C1:  grandfather(terach, X) : {}
-->
C2:  father(terach, F′), father(F′, X) : {}
```

Explanation

- cloned rule: `grandfather(GF’, GS’) :- father(GF’, F’), father(F’, GS’).`
- $\theta' = \{GF'/terach, GS'/X\}$
- new resolvent: `father(GF’, F’), father(F’, GS’)`
- applying θ': `father(terach, F’), father(F’, X)`
- no part of θ' must be recalled for subsequent steps

Prolog resolution by an abstract Scala engine

```scala
trait PrologResolution:
  type Goal
  type Substitution
  type Resolvent = List[Goal]

  case class Clause(head: Goal, body: Resolvent)
  type Program = List[Clause]

  extension (p: Program) def clone: LazyList[Clause]
  extension (r: Resolvent) def apply(s: Substitution): Option[Resolvent]
  def join(s1: Substitution, s2: Substitution): Option[Substitution]
  def mgu(g1: Goal, g2: Goal): Option[Substitution]

  case class State(resolvent: Resolvent, substitution: Substitution)

  extension (program: Program)
    def solve(state: State): LazyList[Substitution] = state match
      case State(goal :: goals, subs) =>
        for
          clause <- program.clone
          mgu <- mgu(goal, clause.head).toSeq
          subs2 <- join(mgu, subs).toSeq
          goals2 <- (clause.body ++ goals).apply(subs2).toSeq
          result <- solve(State(goals2, subs2))
        yield result
      case State(Nil, subs) => LazyList(subs)
```

Logic programming patterns

On Prolog syntax and semantics

- it is essentially all there...
- will now show some useful programming(/design) patterns
- will also show few additional elements (operators/syntax/library)
- will later extend a bit the whole framework

Patterns

- querying facts
- existential queries
- querying universal facts
- working with records
- wilcard variables
- deriving knowledge with rules
- programming math
- programming data types, such as lists

Outline

A Prolog program as a DB

Querying facts

- A Prolog program with only ground facts can be seen as a DB
- All facts of a certain predicate (name + arity) as a table
- A single, ground goal can be used to query the DB for a "tuple"

Existential queries

- if the goal has variables, you are really asking if there exists an instantiation of variables (a substitution) equating your goal with one or more facts
- this is like searching multiple tuples with a query
- composing goals make Prolog find combinations

Universal facts

- variables in facts are quantified universally instead, hence a fact with variables is like an infinite set of facts

Working with records

- a fact can connect terms which are structured, e.g., in the form of records

Use of wildcard variable

- to avoid mentioning a variable once in a clause

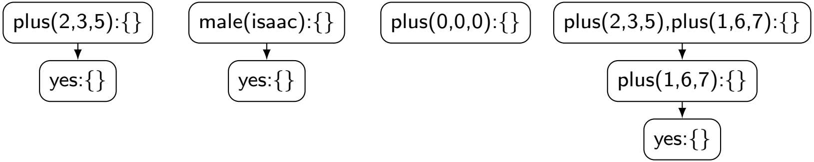
Querying facts

Program

```
male(isaac).
plus(2,3,5).
plus(1,6,7).
plus(0,0,5).
```

Goals

```
?- plus(2,3,5).  Yes:{}
?- male(isaac).  Yes:{}
?- plus(0,0,0).  No
?- plus(2,3,5),plus(1,6,7).  Yes:{}
```



Notes

- "yes" is used to mean empty resolvent (sometimes avoided at all)
- will sometimes avoid ": {}" at all

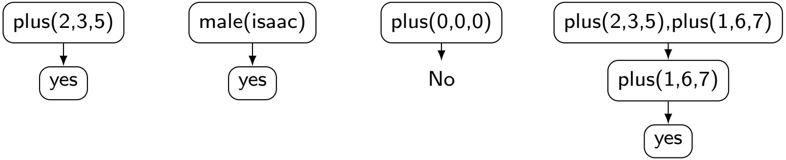
Querying facts: shorter notation for trees

Program

```
male(isaac).
plus(2,3,5).
plus(1,6,7).
plus(0,0,5).
```

Goals

```
?- plus(2,3,5). Yes:{}
?- male(isaac). Yes:{}
?- plus(0,0,0). No
?- plus(2,3,5),plus(1,6,7). Yes:{}
```



Notes

- "yes" is used to mean empty resolvent (sometimes avoided at all)
- when there's no solution will add "no" label
- will sometimes avoid ": {}" at all

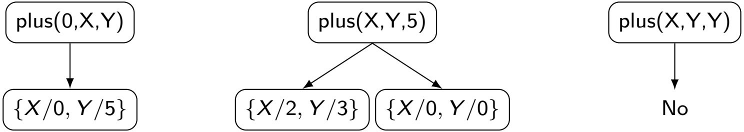
Existential queries

Program

```
plus(2,3,5).
plus(1,6,7).
plus(0,0,5).
```

Goals

```
?- plus(0,X,Y).
     -> {X/0,Y/5}
?- plus(X,Y,5).
     -> {X/2,Y/3}; {X/0,Y/0}
?- plus(X,Y,Y).
     -> No
```



Notes

- recall that we have one branch for rule/fact unifying with the goal
- the unifier is taken as solution

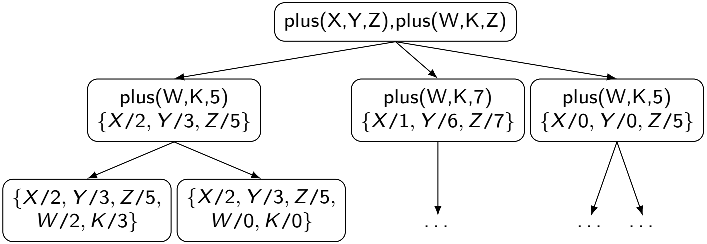
Existential queries and inherent exploration

Program

```
plus(2,3,5).
plus(1,6,7).
plus(0,0,5).
```

Goals

```
?- plus(X,Y,Z),plus(W,K,Z).
    -> {X/2,Y/3,Z/5,W/2,K/3};
    -> {X/2,Y/3,Z/5,W/0,K/0};
    ...
```



Notes

- solving multiple goals inherently explores combinations
- a bit like multiple clauses in for-comprehension

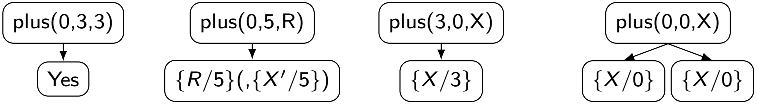
Universal facts

Program

```
plus(0,X,X).
plus(X,0,X).
```

Goals

```
?- plus(0,3,3). -> Yes
?- plus(0,5,R). -> {R/5}
?- plus(3,0,X). -> {X/3} % no clash!
?- plus(0,0,X). -> {X/0}; {X/0}
```



Notes

- the unification with a universal fact creates a substitution mentioning variables of a cloned clause (see the (,{X'/5}))
- however, they are not showed as final result

Working with records

Program

```
manager(person(john,smith,1283)).
clerk(person(jim,white,3475)).
clerk(person(george,red,8765)).
chief(person(john,smith,1283), person(george,red,8765)).
```

Goal

```
?-chief(X,person(Y,Z,8765)).
yes.
X/person(john,smith,1283) Y/george Z/red
chief(person(john,smith,1283), person(george,red,8765)).
```

Notes

- usage of facts as relations (1-ary, 2-ary) between records
- recall `manager` is a predicate, `person` a functor
- note Prolog outputs minimal substitution and instantiated goal.

Working with records

Program

```
manager(person(john,smith,1283)).
clerk(person(jim,white,3475)).
clerk(person(george,red,8765)).
chief(person(john,smith,1283), person(george,red,8765)).
```

Goal

```
?-chief(X,person(Y,Z,8765)).
yes.
X/person(john,smith,1283) Y/george Z/red
chief(person(john,smith,1283), person(george,red,8765)).
```

Notes

- usage of facts as relations (1-ary, 2-ary) between records
- recall manager is a predicate, person a functor
- note Prolog outputs minimal substitution and instantiated goal.

Wildcard variable

Program

```
p(_,1).
p(1,2).
q(_,a(1,_)).
r(X,a(1,X)).
```

Goals

```
?- p(X,1). Yes
?- p(1,Y). {Y/1}; {Y/2}
?- p(1,_). Yes; Yes
?- q(1,a(1,2)). Yes
?- r(1,a(1,2)). No
```

Semantics

- in programs: a variable used once in the clause
- in goals: a variable we do not need to occur in solutions

Notes

- it is good Prolog practice to never use singleton variables in a clause, but wildcards instead
- this is used to avoid copy-and-paste errors in variable names

A recap example: modelling propositional logic

Goals

```
b_not(b_true, b_false).
b_not(b_false, b_true).
b_and(B, b_true, B).
b_and(B, b_false, b_false).
b_or(B, b_false, B).
b_or(B, b_true, b_true).
```

Goals

```
?- b_not(b_false, B).
     -> {B/b_true}
?- b_and(b_false, b_true, B).
     -> {B/b_false}
?- b_or(b_false, b_true, B).
     -> {B/b_true}
?- b_or(b_true, B2, B). -> ???
?- b_or(B1, B2, B). -> ???
```

Modelling idea: this is a simple data type!

- we have two booleans, modelled as atomic terms `b_true, b_false` (functors with arity 0)
- a unary function is modelled as a binary `p(I,O)` predicate
- a binary function is modelled as a ternary `p(I1,I2,O)` predicate
- use universal facts to somewhat address DRY

⇒ can we derive a general approach for data types?

Outline

Programming data types

Definition of a data type

- Construction + algorithms
- Construction defined by deciding functors (name + arity) for terms
- Algorithms by predicates (with function to relation mapping)

Example: programmed booleans

- improvement with rules

Example: booleans with built-in engine

- functions yielding a boolean as a predicate

Example: Peano arithmetics

- showcasing recursive rules, and recursion

Example: lists as recursive data types

- a playground for exercises

Algebraic data types in Prolog

Recall features of an (albegraic) data type

- A name of the type
- A set of values, expressed by "sums" of "products"
- A set of I/O pure functions: often using recursion

In Prolog:

We have no types, hence no enforcement

- Name: can only impact names of values/functions
- Values:
  - in terms of a set of functors, e.g.: (`cons/2` and `nil/0`)
- Functions:
  - $f : I_1, I_2, \ldots, I_m \mapsto O$ becomes `p(I1,I2,...,In,O)`
  - define functions (in the body) as composition of (goals) relations
  - `h(X)=f(g(X))` becomes `h(X,Y) :- g(X,Z), f(Z,Y).`
  - hopefully using matching on functors
  - functions returning booleans might have special treatment

Example 1: Improving booleans with rules

```
b_not(b_true, b_false).
b_not(b_false, b_true).
b_and(B, b_true, B).
b_and(B, b_false, b_false).
b_or(B1, B2, B) :-           % (a or b) = !(!a and !b)
    b_not(B1, NB1), b_not(B2, NB2),  b_and(NB1, NB2, NB), b_not(NB, B).
b_implies(B1, B2, B) :-      % a --> b = !a or b
    b_not(B1, NB1), b_or(NB1, B2, B).
```

b_implies(b_true, b_false,O)

↓

b_not(b_true, NB1'), b_or(NB1', b_false, O)

↓

b_or(b_false, b_false, O)

↓

b_not(b_false, NB1'),b_not(b_false, NB2'),b_and(NB1', NB2', NB'),b_not(NB', O)

↓

b_not(b_false, NB2'),b_and(b_true, NB2', NB'),b_not(NB', O)

↓

b_and(b_true, b_true, NB'),b_not(NB', O)

↓

b_not(b_true, O)

↓

$\{O/b\_false\}$

Example 2: naturals by "Peano numbers"

Encoding natural numbers

- traditionally, the next step after booleans while studying a language expressiveness
- an easy encoding is the unary, Peano one: Z,SZ,SSZ,SSSZ,SSSSZ,...
- hence we would have, e.g.: SSZ + SSSZ = SSSSSZ
- Prolog will actually have an ad-hoc management of numbers (and booleans)

Ideas

- use functors `s/1` and `zero/0`, e.g.: `s(s(s(zero)))`
- `succ` function is easily obtained by `s/1` construction
- `sum` function is obtained recursively from `succ` or `s/1`
  - $a + 0 = a$, $a + s(b) = s(a + b)$
- `mul` function is obtained recursively from `sum`
  - $a * 0 = 0$, $a * s(b) = a + (a * b)$
- e.g., we could be able to implement `factorial`
- output is last argument as usual

Example 2: solution

```
succ(X, s(X)).
sum(X, zero, X).
sum(X, s(Y), s(Z)) :- sum(X, Y, Z).
mul(X, zero, zero).
mul(X, s(Y), Z) :- mul(X, Y, W), sum(W, X, Z).
dec(s(X), X).
factorial(zero, s(zero)).
factorial(s(X), Y):-factorial(X, Z), mul(s(X), Z, Y).
```

How to read the specification in the relational interpretation

- the successor of `X` is `s(X)`
- the sum of `X` and `zero` is `X`
- the sum of `X` and successor of `Y` is the successor of `Z` provided the sum of `X` and `Y` is `Z`
- etcetera...

Example 2: resolution

```
succ(X, s(X)).
sum(X, zero, X).
sum(X, s(Y), s(Z)) :- sum(X, Y, Z).
mul(X, zero, zero).
mul(X, s(Y), Z) :- mul(X, Y, W), sum(W, X, Z).
dec(s(X), X).
factorial(zero, s(zero)).
factorial(s(X), Y):-factorial(X, Z), mul(s(X), Z, Y).
```

```
?- succ(s(s(zero)), N). -> N/s(s(s(zero)))
?- sum(s(s(s(zero))), s(s(zero)), N). -> N/s(s(s(s(s(zero)))))
?- mul(s(s(zero)), s(s(zero)), N). -> N/s(s(s(s(zero))))
?- dec(s(s(zero)), N). -> N/s(zero)
?- dec(zero), N). -> No
?- sum(N, M, s(s(s(zero)))). -> ???
```

sum(s(s(s(zero))), s(s(zero)), N)

sum(s(s(s(zero))), s(zero), Z') : $\{N/s(Z')\}$

sum(s(s(s(zero))), zero, Z") : $\{N/s(Z'), Z'/s(Z'')\} \equiv \{N/s(s(Z''))\}$

$\{N/s(Z'), Z'/s(Z''), Z''/s(s(s(zero)))\} \equiv \{N/s(s(s(s(s(zero)))))\}$

Example 2: compare to modern FP solution

```
succ(X, s(X)).
sum(X, zero, X).
sum(X, s(Y), s(Z)) :- sum(X, Y, Z).
mul(X, zero, zero).
mul(X, s(Y), Z) :- mul(X, Y, W), sum(W, X, Z).
dec(s(X), X).
factorial(zero, s(zero)).
factorial(s(X), Y):-factorial(X, Z), mul(s(X), Z, Y).
```

```
enum Nat:
  case Zero
  case S(n: Nat)

object Nat:
  def succ(n: Nat): Nat = S(n)
  def sum(n1: Nat, n2: Nat): Nat = n2 match
    case Zero => n1
    case S(n) => S(sum(n1, n))
  def mul(n1: Nat, n2: Nat): Nat = n2 match
    case  Zero => Zero
    case S(n) => sum(n1, mul(n1, n))

@main def mainPeano() =
  import Nat.*
  val one = S(Zero)
  val two = S(S(Zero))
  val three = S(S(S(Zero)))
  println(succ(one))
  println(sum(two, three))
  println(mul(two, three))
```

Dealing with functions returning a boolean

Typical Prolog approach

- Not to have an additional argument being `b_true` or `b_false`, but rather...
- $f : I_1, I_2, \ldots, I_m \mapsto bool$ becomes `p(I1,I2,...,In)`, the result is whether call to the predicate fails or succeeds
- in fact, one such function is a predicate
- this way, we ca only implement what should happen in positive cases

```
greater(s(_), zero).
greater(s(N), s(M)) :- greater(N, M).
```

```
?- greater(s(zero), s(zero)). -> No
?- greater(s(s(zero)), s(zero)). -> Yes
```

Dealing with multiple output arguments

Typical Prolog approach

- Not to have an output of type `Pair`, but rather...
- $f : I_1, I_2, \ldots, I_m \mapsto O1 \times O2$ becomes `p(I1,I2,...,In,O1,O2)`
- in fact, we can conceptually have many inputs and many outputs

```
nextprev(s(N), N, s(s(N))).
```

```
?- nextprev(s(s(zero)), Prev, Next)).
    -> {Prev/s(zero), Next/s(s(s(zero)))
?- nextprev(zero, Prev, Next)).
    -> No
```

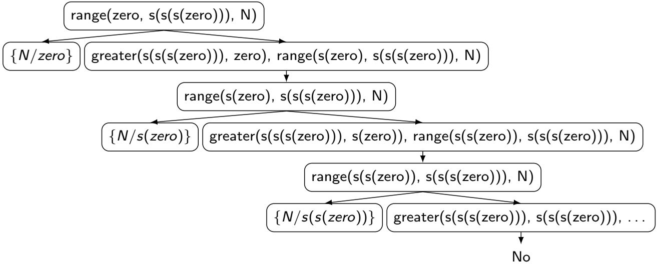
Dealing with multiple results

Typical Prolog approach

- Not to have a result as a sort of lazy list, but rather...
- $f : I_1, I_2, \ldots, I_m \mapsto LazyList$ becomes `p(I1,I2,...,In,O)` such that it provides multiple solutions due to the inherent Prolog resolution mechanism
- in fact, we should always be prepared that goals admit many solutions

```
range(N1, N2, N1).
range(N1, N2, N) :- greater(N2, N1), range(s(N1), N2, N).
```

```
?- range(zero, s(s(s(zero))), N).
    -> {N/zero}; {N/s(zero)}; {N/s(s(zero))}
```



Example 3: (linked)lists by `cons/nil` construction

Ideas

- use functors `cons/2` (with head and tail as args) and `nil/0` for empty lists
- e.g.: `cons(a, nil)`, `cons(a, cons(b, nil))` – note they are trees
- generally implement functions/predicates by matching, through different clauses
- recursive functions by base cases implemented as facts, and then by recursive rules

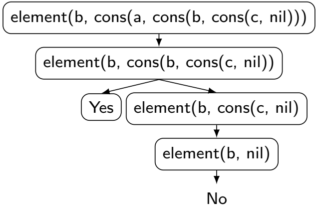
Example 3: element over lists

```
% relates an element E with a list that contains it
element(E, cons(E, _)).
element(E, cons(_, T)) :- element(E, T).
```

How to read the specification in the relational interpretation

- E is found in a list with head E
- E is found in a list with tail T provided E is found in T

```
?- element(b, cons(a, cons(b, cons(c, nil)))). -> Yes; No
?- element(a, cons(a, cons(b, cons(c, nil)))). -> Yes; No
?- element(40, cons(a, cons(b, cons(c, nil)))). -> No
```



Programming find/position/concat/count

```
% relates a list with one of its elements
find(cons(E, _), E).
find(cons(_, T),E) :- find(T, E).

% relates a list with with one of its elements and its Peano position
position(cons(E, _), zero, E).
position(cons(H, T), s(N), E) :- position(T, N, E).

% relates two lists with their concatenation (similar to append)
concat(nil, L, L).
concat(cons(H, T), L, cons(H, M)):- concat(T, L, M).

% relates a list and an element with occurrences
count(nil, E, zero).
count(cons(E, L), E, s(N)) :- count(L, E, N).
count(cons(E, L), E2, N) :- E \= E2, count(L, E2, N).
```

```
?- find(cons(a, cons(b, cons(c, nil))), b). -> Yes
?- find(cons(a, cons(b, cons(c, nil))), d). -> No
?- position(cons(a, cons(b, cons(c, nil))), zero, a). -> Yes1
?- position(cons(a, cons(b, cons(c, nil))), s(zero), b). -> Yes
?- position(cons(a, cons(b, cons(b, nil))), P, b).  -> P/s(zero); P/s(s(zero))
?- concat(cons(a, cons(b, cons(c, nil))), cons(d, nil), L).
     -> L/cons(a, cons(b, cons(c, cons(d, nil))))
?- count( cons(a, cons(a, cons(b, cons(a, nil)))), a, N). -> N/s(s(s(zero)))
```

Next steps

Expressiveness so far

- conceptually "complete"
- can implement a variety of algorithms to search/clone/modify algebraic-like data structures

Features to be added, in next lessons

- more convenient ways to work with numbers (primitive values)
- more convenient ways to work with lists
- library functions to manipulate terms
- library functions to tweak resolution